TECNOLÓGICO NACIONAL DE MÉXICO

INSTITUTO TECNOLÓGICO DE CULIACÁN



TECNOLÓGICO NACIONAL DE MÉXICO



ACTIVIDAD

Informe del Proyecto Ensamblador de la Unidad 2

CARRERA:

Ingeniería en Sistemas Computacionales

MATERIA:

Lenguajes de Interfaz

Alumnos:

López Payán Jorge Emilio

Sotelo Izaguirre José Ángel

DOCENTE:

Medina Jiménez José Luis

FECHA:

Culiacán, Sinaloa, a 30 de octubre de 2026

ÍNDICE

Negocio Elegido..... 3
Regla adicional.....4
Organización de datos en memoria..... 5
Formato de del archivo..... 7
Referencias Bibliográficas.....8

Negocio Elegido

El negocio elegido para desarrollar el proyecto es una librería en línea inspirada en tiendas como Librerías Gandhi. El catálogo, como se indica en la rúbrica del proyecto, incluye los títulos de los libros, un id con tres letras y tres dígitos, precio del libro y cantidad de libros en existencias. La razón por la que se eligió este negocio radica en que una tienda en línea de este calibre opera con un catálogo amplio de libros y secciones, lo cual se asemeja a lo que pide principalmente el proyecto: registrar productos, recibir mercancía y vender.

El programa administra la máxima cantidad de 10 productos. Cada uno de ellos cuenta con un formato de id en específico, un nombre de máximo 25 caracteres, un precio de máximo $9999.99 y una cantidad de existencias de 100 unidades como máximo; estas capturas se realizan con ReadString y las conversiones numéricas se realizan aparte.

Regla adicional

La regla adicional elegida fue la siguiente: En una venta de un solo producto, si la cantidad es equivalente a 5 o más ejemplares, se descontará un 10% del importe base. Si se tiene abajo de 5 ejemplares, no habrá descuento. Su cálculo se realiza de la siguiente manera:

- Importe base = precio en centavos × cantidad.
- Descuento = importe base × 10 ÷ 100, descartando la fracción de centavo.
- Importe final = importe base − descuento. Este importe es el que se suma a los ingresos de la sesión.

Organización de datos en memoria

Los productos se guardan en un solo arreglo de bytes llamado *tabla*, el cual tiene la extensión de 880 bytes equivalentes a 20 registros de de 44 bytes; se usa un espacio de 20 registros debido a que los primeros 440 bytes contienen el catálogo activo, mientras que los otros 440 bytes corresponden a una “zona temporal” cuyo uso radica en almacenar valores temporales en lo que se valida el archivo antes de reemplazar el catálogo. Se destinaron 44 bytes para cada registro debido a la siguiente organización de bytes mostrada en la tabla:

Desplazamiento	Atributo del artículo	Tamaño
0	id (7 caracteres + nulo)	8 bytes
+8	nombre (hasta 25 caracteres + nulo)	28 bytes
+36	precio	DWORD
+40	cantidad	DWORD

Para localizar un producto ESI guarda la posición del registro dentro de *tabla* y avanza con *add esi, 44*. Con este incremento es posible acceder a cada campo, su demostración quedaría de la siguiente manera:

- tabla[esi] corresponde al carácter del id
- tabla[esi+36] corresponde al precio
- tabla[esi+40] corresponde a la cantidad, en este caso se usa DWORD PTR cuando se leen 4 bytes.

BuscarId recibe en ESI en donde empezar, se usa 0 para indicar el catálogo y 440 para indicar la zona temporal, por lo que se usa para revisar ids duplicados tanto al dar de alta como al cargar el archivo.

Otro aspecto fundamental que usa datos en memoria es el precio en centavos; para este procedimiento el precio se convierte a centavos enteros al capturarlos, por ejemplo 185.85 pasa a ser 18585 y así se guarda. Para mostrarlo se utiliza el método *MostrarMXN* el cual divide entre 100, el cociente son los pesos y el residuo son los centavos; si el residuo es menor que 10 entonces se antepone un 0.

A lo largo del programa se hizo uso de una considerable cantidad de variables y de buffers, los principales utilizados fueron los siguientes:

- *totalProd*: Corresponde a la variable de los productos registrados.
- *tmpTotal*: Corresponde a la variable de los productos leídos del archivo.
- *ingresos*: Corresponde a la variable que almacena los ingresos de la sesión en curso.

- *cambios*: Corresponde a una variable que se actualiza al valor de 1 en caso de haya cambios sin guardar o 0 si no hay cambios, ayuda a conservar el estado del programa.
- *entrada*: Corresponde a un buffer de 40 bytes que lee cadenas, todas las validaciones y conversiones trabajan sobre él utilizando ESI como índice.
- *numTexto*: Corresponde a un buffer de 12 bytes que almacena una cadena decimal de un número.
- *bufArchivo*: Corresponde a un buffer de 513 bytes que ayuda a la lectura de archivos
- *bufSalida*: Corresponde a un buffer de 600 bytes que ayuda a escribir el texto que se escribe al guardar.

Por último en este apartado se debe de especificar cuáles fueron las validaciones y desbordamiento utilizadas en términos generales. Primero, los datos se escriben en el primer registro libre mientras se capturan, pero el producto solo cuenta cuando todo es válido, porque *totalProd* se incrementa hasta el final; si algo falla, el registro queda sin contarse y el catálogo no cambia. Segundo, en una venta primero se validan existencias y cálculos, se deja para el final la actualización de existencias e ingreso de los datos. En tercer lugar se tiene el desbordamiento, este detecta con *MUL* (si EDX es distinto a 0, el resultado no cabe en 32 bits) *y con ADD* (seguido de JC, el acarreo).

Formato de del archivo

Se utilizó un archivo .txt como se enseñó en clase, el archivo mantiene un dato por línea y estas están concluidas con *CRLF* (o *0Dh 0Ah* en su defecto). La primera línea es el total de productos y luego, por cada producto, se tienen cuatro líneas correspondientes a los atributos de los libros. El archivo de texto se puede apreciar de la siguiente manera:

```
5
LIB-123
EL EXTRANJERO
9999
90
LIB-321
1984
35000
75
LIB-231
PLATAFORMA
15000
10
LIB-777
WIGETTA
41194
5
LIB-666
NOCHES BLANCAS
6000
89
```

Para el procedimiento de guardado, el programa arma todo en *bufSalida* (los números previamente se convierten con *convertirDecimal*) y lo escribe con *CreateOutputFile*, *WriteToFile* y *CloseFile*. Para el guardado se comprueba que el handle sea válido, que no se tenga un acarreo, que los bytes escritos sean equivalentes a los solicitados y que el cierre concluya exitosamente.

Para el procedimiento de carga, el programa hace uso de *OpenInputFile* y se lee con *ReadFromFile* (el cual tiene una capacidad de 512 bytes), después de esto se agrega el terminador nulo justo después del último byte leído. Si el contenido leído llena el buffer (512 bytes), se considera que el archivo es demasiado grande o pudo quedar incompleto, por lo que se rechaza sin modificar el catálogo. El tamaño máximo aceptado es de 511 bytes. Los datos son validados en la parte de temporales de tabla y solo se copian al catálogo si todo es válido.

Si el archivo no existe o no es válido por algún daño, se muestra un mensaje referente a esto y el catálogo mantiene su estado actual.

Referencias Bibliográficas

Ferrari, A., Batson, A., Lack, M., Jones, A. y Evans, D. (2006, 10 de abril). x86 assembly guide [Material de curso]. CS216: Program and Data Representation, University of Virginia. https://www.cs.virginia.edu/~evans/cs216/guides/x86.pdf

Iqbal, K. (2019). TXT - Archivo de documento de texto. FileFormat.com. https://docs.fileformat.com/es/word-processing/txt/